EXPERIMENT-03

ID3 Decision Tree Algorithm: Dataset-Based Python Programs

Name: B.ANJALI Reg.No: 192371062

3. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Problem 1: Playing Tennis

Aim

To demonstrate the ID3 decision tree algorithm on the PlayTennis dataset stored in a CSV file and classify a new weather sample.

Procedure

1. Save the training examples in playtennis_data.csv with the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and calculate the entropy of the class column.
4. Calculate the information gain of every attribute and choose the attribute with the highest gain as the root node.
5. Split the data on that attribute and repeat the process on each branch until a branch contains only one class.
6. Print the decision tree and use it to classify a new sample.

Dataset File / Input

File name: playtennis_data.csv

The CSV file contains a header row followed by these training examples:

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Code

```
import json
import numpy as np
import pandas as pd

df = pd.read_csv("playtennis_data.csv")
target = df.columns[-1]

def entropy(col):
    p = col.value_counts(normalize=True)
```

```
    return -(p * np.log2(p)).sum()

def gain(d, a):
    return entropy(d[target]) - sum(len(s) / len(d) * entropy(s[target]) for _, s in d.groupby(a))

def id3(d, attrs):
    if d[target].nunique() == 1:                    # pure node -> leaf
        return d[target].iloc[0]
    if not attrs:                                   # no attributes left -> majority class
        return d[target].mode()[0]
    best = max(attrs, key=lambda a: gain(d, a))     # attribute with highest gain
    return {best: {v: id3(s, [a for a in attrs if a != best]) for v, s in d.groupby(best)}}

def classify(tree, x):
    while isinstance(tree, dict):
        a = next(iter(tree))
        tree = tree[a].get(x[a], "Unknown")
    return tree

attrs = list(df.columns[:-1])
print("Information Gain:", {a: round(float(gain(df, a)), 3) for a in attrs})
tree = id3(df, attrs)
print("Decision Tree:", json.dumps(tree, indent=2))

new = {"Outlook": "Sunny", "Temperature": "Cool", "Humidity": "High", "Wind": "Strong"}
print("New sample:", new)
print("Prediction:", classify(tree, new))
```

Output

```
Information Gain: {'Outlook': 0.247, 'Temperature': 0.029, 'Humidity': 0.152, 'Wind': 0.048}
Decision Tree: {
  "Outlook": {
    "Overcast": "Yes",
    "Rain": {
      "Wind": {
        "Strong": "No",
        "Weak": "Yes"
      }
    },
    "Sunny": {
      "Humidity": {
        "High": "No",
        "Normal": "Yes"
      }
    }
  }
}
New sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Prediction: No
```

Result

Outlook has the highest information gain (0.247), so it becomes the root node. Overcast always leads to Yes, Sunny is split on Humidity and Rain is split on Wind. The new sample (Sunny, Cool, High, Strong) follows Outlook = Sunny and Humidity = High, so it is classified as PlayTennis = No.

Problem 2: Buying a Computer

Aim

To build an ID3 decision tree for a computer-purchase dataset stored in a CSV file and use it to classify a new customer.

Procedure

1. Save the training examples in computer_data.csv with the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and calculate the entropy of the class column.
4. Calculate the information gain of every attribute and choose the attribute with the highest gain as the root node.

5. Split the data on that attribute and repeat the process on each branch until a branch contains only one class.
6. Print the decision tree and use it to classify a new sample.

Dataset File / Input

File name: computer_data.csv

The CSV file contains a header row followed by these training examples:

Age	Income	Student	CreditRating	BuysComputer
Youth	High	No	Fair	No
Youth	High	No	Excellent	No
Youth	Medium	No	Fair	Yes
Middle	Low	No	Fair	Yes
Middle	Medium	No	Excellent	No
Senior	Medium	No	Fair	Yes
Senior	Low	No	Excellent	No
Youth	Medium	Yes	Fair	Yes
Youth	Low	Yes	Excellent	Yes
Middle	Low	Yes	Fair	Yes
Middle	Medium	Yes	Excellent	Yes
Senior	Low	Yes	Fair	Yes
Senior	Medium	Yes	Excellent	Yes
Middle	High	Yes	Excellent	Yes

Code

```
import json
import numpy as np
import pandas as pd

df = pd.read_csv("computer_data.csv")
target = df.columns[-1]

def entropy(col):
    p = col.value_counts(normalize=True)
    return -(p * np.log2(p)).sum()

def gain(d, a):
    return entropy(d[target]) - sum(len(s) / len(d) * entropy(s[target]) for _, s in d.groupby(a))

def id3(d, attrs):
    if d[target].nunique() == 1:                    # pure node -> leaf
        return d[target].iloc[0]
    if not attrs:                                   # no attributes left -> majority class
        return d[target].mode()[0]
    best = max(attrs, key=lambda a: gain(d, a))     # attribute with highest gain
    return {best: {v: id3(s, [a for a in attrs if a != best]) for v, s in d.groupby(best)}}

def classify(tree, x):
    while isinstance(tree, dict):
        a = next(iter(tree))
        tree = tree[a].get(x[a], "Unknown")
    return tree

attrs = list(df.columns[:-1])
print("Information Gain:", {a: round(float(gain(df, a)), 3) for a in attrs})
tree = id3(df, attrs)
print("Decision Tree:", json.dumps(tree, indent=2))

new = {"Age": "Senior", "Income": "Low", "Student": "No", "CreditRating": "Fair"}
print("New sample:", new)
print("Prediction:", classify(tree, new))
```

Output

```
Information Gain: {'Age': 0.027, 'Income': 0.13, 'Student': 0.371, 'CreditRating': 0.075}
Decision Tree: {
  "Student": {
    "No": {
      "CreditRating": {
        "Excellent": "No",
        "Fair": {
          "Income": {
            "High": "No",
            "Low": "Yes",
            "Medium": "Yes"
          }
        }
      }
    },
    "Yes": "Yes"
  }
}
New sample: {'Age': 'Senior', 'Income': 'Low', 'Student': 'No', 'CreditRating': 'Fair'}
Prediction: Yes
```

Result

Student has the highest information gain (0.371) and is the root node: every student buys a computer. Non-students are split on CreditRating (Excellent gives No) and then on Income when the rating is Fair. The new sample (Senior, Low, No, Fair) follows Student = No, CreditRating = Fair and Income = Low, so it is classified as BuysComputer = Yes.

Problem 3: Loan Approval

Aim

To apply the ID3 algorithm to a loan-approval dataset stored in a CSV file and classify a new applicant.

Procedure

1. Save the training examples in loan_data.csv with the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and calculate the entropy of the class column.
4. Calculate the information gain of every attribute and choose the attribute with the highest gain as the root node.
5. Split the data on that attribute and repeat the process on each branch until a branch contains only one class.
6. Print the decision tree and use it to classify a new sample.

Dataset File / Input

File name: loan_data.csv

The CSV file contains a header row followed by these training examples:

Income	CreditScore	Employment	Property	LoanApproved
High	Good	Salaried	Yes	Yes
High	Good	SelfEmployed	No	Yes
Medium	Good	Salaried	Yes	Yes
Medium	Good	Unemployed	No	No
Low	Good	Salaried	No	Yes
High	Average	Salaried	Yes	Yes
Medium	Average	SelfEmployed	No	No
Low	Average	Salaried	No	No
High	Poor	Salaried	Yes	No
Medium	Poor	SelfEmployed	No	No
Low	Poor	Unemployed	No	No
Medium	Average	Salaried	No	Yes
Low	Good	Unemployed	No	No
High	Average	SelfEmployed	No	Yes

Code

```
import json
import numpy as np
import pandas as pd

df = pd.read_csv("loan_data.csv")
target = df.columns[-1]

def entropy(col):
    p = col.value_counts(normalize=True)
    return -(p * np.log2(p)).sum()

def gain(d, a):
    return entropy(d[target]) - sum(len(s) / len(d) * entropy(s[target]) for _, s in d.groupby(a))

def id3(d, attrs):
    if d[target].nunique() == 1:                    # pure node -> leaf
        return d[target].iloc[0]
    if not attrs:                                   # no attributes left -> majority class
        return d[target].mode()[0]
    best = max(attrs, key=lambda a: gain(d, a))     # attribute with highest gain
    return {best: {v: id3(s, [a for a in attrs if a != best]) for v, s in d.groupby(best)}}

def classify(tree, x):
    while isinstance(tree, dict):
        a = next(iter(tree))
        tree = tree[a].get(x[a], "Unknown")
    return tree

attrs = list(df.columns[:-1])
print("Information Gain:", {a: round(float(gain(df, a)), 3) for a in attrs})
tree = id3(df, attrs)
print("Decision Tree:", json.dumps(tree, indent=2))

new = {"Income": "Medium", "CreditScore": "Good", "Employment": "SelfEmployed", "Property": "Yes"}
print("New sample:", new)
print("Prediction:", classify(tree, new))
```

Output

```
Information Gain: {'Income': 0.164, 'CreditScore': 0.26, 'Employment': 0.283, 'Property': 0.075}
Decision Tree: {
  "Employment": {
    "Salaried": {
      "CreditScore": {
        "Average": {
          "Income": {
            "High": "Yes",
            "Low": "No",
            "Medium": "Yes"
          }
        },
        "Good": "Yes",
        "Poor": "No"
      }
    },
    "SelfEmployed": {
      "Income": {
        "High": "Yes",
        "Medium": "No"
      }
    },
    "Unemployed": "No"
  }
}
New sample: {'Income': 'Medium', 'CreditScore': 'Good', 'Employment': 'SelfEmployed', 'Property': 'Yes'}
Prediction: No
```

Result

Employment has the highest information gain (0.283) and is the root node. Unemployed applicants are rejected, salaried applicants are decided by CreditScore (and by Income when the score is Average), and self-employed applicants are decided by Income. The new sample (Medium, Good, SelfEmployed, Yes) follows Employment = SelfEmployed and Income = Medium, so it is classified as LoanApproved = No.

Problem 4: Student Result

Aim

To build an ID3 decision tree that predicts whether a student passes or fails from attendance and study details stored in a CSV file, and classify a new student.

Procedure

1. Save the training examples in student_data.csv with the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and calculate the entropy of the class column.
4. Calculate the information gain of every attribute and choose the attribute with the highest gain as the root node.
5. Split the data on that attribute and repeat the process on each branch until a branch contains only one class.
6. Print the decision tree and use it to classify a new sample.

Dataset File / Input

File name: student_data.csv

The CSV file contains a header row followed by these training examples:

Attendance	Assignments	StudyHours	PreviousResult	Result
High	Done	High	Good	Pass
High	Pending	Low	Poor	Pass
High	Done	Medium	Poor	Pass
High	Pending	Medium	Good	Pass
High	Done	Low	Good	Pass
Low	Done	High	Good	Pass
Low	Done	Low	Poor	Pass
Low	Pending	Medium	Good	Fail
Low	Pending	Low	Poor	Fail
Low	Pending	High	Good	Fail
Low	Done	Medium	Poor	Pass
Low	Pending	High	Poor	Fail

Code

```
import json
import numpy as np
import pandas as pd

df = pd.read_csv("student_data.csv")
target = df.columns[-1]

def entropy(col):
    p = col.value_counts(normalize=True)
    return -(p * np.log2(p)).sum()

def gain(d, a):
    return entropy(d[target]) - sum(len(s) / len(d) * entropy(s[target]) for _, s in d.groupby(a))

def id3(d, attrs):
    if d[target].nunique() == 1:                    # pure node -> leaf
        return d[target].iloc[0]
    if not attrs:                                   # no attributes left -> majority class
        return d[target].mode()[0]
    best = max(attrs, key=lambda a: gain(d, a))     # attribute with highest gain
    return {best: {v: id3(s, [a for a in attrs if a != best]) for v, s in d.groupby(best)}}
```

```
def classify(tree, x):
    while isinstance(tree, dict):
        a = next(iter(tree))
        tree = tree[a].get(x[a], "Unknown")
    return tree

attrs = list(df.columns[:-1])
print("Information Gain:", {a: round(float(gain(df, a)), 3) for a in attrs})
tree = id3(df, attrs)
print("Decision Tree:", json.dumps(tree, indent=2))

new = {"Attendance": "High", "Assignments": "Pending", "StudyHours": "Low", "PreviousResult": "Good"}
print("New sample:", new)
print("Prediction:", classify(tree, new))
```

Output

```
Information Gain: {'Attendance': 0.344, 'Assignments': 0.459, 'StudyHours': 0.044, 'PreviousResult': 0.0}
Decision Tree: {
  "Assignments": {
    "Done": "Pass",
    "Pending": {
      "Attendance": {
        "High": "Pass",
        "Low": "Fail"
      }
    }
  }
}
New sample: {'Attendance': 'High', 'Assignments': 'Pending', 'StudyHours': 'Low', 'PreviousResult': 'Good'}
Prediction: Pass
```

Result

Assignments has the highest information gain (0.459) and is the root node. Students with completed assignments pass; when assignments are pending, Attendance decides (High gives Pass, Low gives Fail). StudyHours and PreviousResult are not needed in the tree. The new sample (High, Pending, Low, Good) is classified as Result = Pass.

Problem 5: Mushroom Edibility

Aim

To apply the ID3 algorithm to a mushroom-edibility dataset stored in a CSV file and classify a new mushroom sample.

Procedure

1. Save the training examples in mushroom_data.csv with the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and calculate the entropy of the class column.
4. Calculate the information gain of every attribute and choose the attribute with the highest gain as the root node.
5. Split the data on that attribute and repeat the process on each branch until a branch contains only one class.
6. Print the decision tree and use it to classify a new sample.

Dataset File / Input

File name: mushroom_data.csv

CapColor	Odor	Gills	Ring	Edible
Brown	NoSmell	Broad	Yes	Yes
White	NoSmell	Narrow	Yes	Yes
Red	Foul	Broad	No	No
Brown	Foul	Narrow	No	No
White	Almond	Broad	Yes	Yes
Red	NoSmell	Narrow	No	Yes
Brown	Almond	Narrow	Yes	Yes
White	Foul	Broad	Yes	No

Red	Almond	Broad	No	Yes
Brown	NoSmell	Narrow	No	Yes
White	Foul	Narrow	No	No
Red	Foul	Narrow	Yes	No

The CSV file contains a header row followed by these training examples:

Code

```
import json
import numpy as np
import pandas as pd

df = pd.read_csv("mushroom_data.csv")
target = df.columns[-1]

def entropy(col):
    p = col.value_counts(normalize=True)
    return -(p * np.log2(p)).sum()

def gain(d, a):
    return entropy(d[target]) - sum(len(s) / len(d) * entropy(s[target]) for _, s in d.groupby(a))

def id3(d, attrs):
    if d[target].nunique() == 1:                    # pure node -> leaf
        return d[target].iloc[0]
    if not attrs:                                   # no attributes left -> majority class
        return d[target].mode()[0]
    best = max(attrs, key=lambda a: gain(d, a))     # attribute with highest gain
    return {best: {v: id3(s, [a for a in attrs if a != best]) for v, s in d.groupby(best)}}

def classify(tree, x):
    while isinstance(tree, dict):
        a = next(iter(tree))
        tree = tree[a].get(x[a], "Unknown")
    return tree

attrs = list(df.columns[:-1])
print("Information Gain:", {a: round(float(gain(df, a)), 3) for a in attrs})
tree = id3(df, attrs)
print("Decision Tree:", json.dumps(tree, indent=2))

new = {"CapColor": "Brown", "Odor": "Foul", "Gills": "Broad", "Ring": "Yes"}
print("New sample:", new)
print("Prediction:", classify(tree, new))
```

Output

```
Information Gain: {'CapColor': 0.043, 'Odor': 0.98, 'Gills': 0.001, 'Ring': 0.021}
Decision Tree: {
  "Odor": {
    "Almond": "Yes",
    "Foul": "No",
    "NoSmell": "Yes"
  }
}
New sample: {'CapColor': 'Brown', 'Odor': 'Foul', 'Gills': 'Broad', 'Ring': 'Yes'}
Prediction: No
```

Result

Odor has by far the highest information gain (0.98), so one split classifies every training example: Foul gives No, while Almond and NoSmell give Yes. The new sample (Brown, Foul, Broad, Yes) is classified as Edible = No.